



ITI · PYTHON FRAMEWORK WITH FASTAPI TRACK

Python Framework with FastAPI

Eng. Fatma Elraey*Lecture 4 — Simple Alembic Migrations, APIRouter & API Documentation*

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lecture 4 of 5
Focus	Version one SQLite schema change with Alembic, organize the same Session(engine) CRUD routes with APIRouter, and inspect FastAPI's generated API docs.

Welcome!

This session continues directly from the simplified SQLite ORM API in Lecture 3. We keep the same documents.db file and the same direct with Session(engine) as db pattern. The new ideas are how to version one database schema change, move routes out of main.py, and use FastAPI's generated API documentation.

Today's Outline

1. Why schema changes need migrations
2. Connect Alembic to the simple SQLite ORM project
3. Create and apply one migration
4. Organize Documents routes with APIRouter
5. Swagger UI, ReDoc, OpenAPI and documented responses

01 Why Schema Changes Need Migrations

In Lecture 3, `Base.metadata.create_all(engine)` was enough to create the documents table when it was missing. It is not a history of later schema changes. If `documents.db` already exists and we add a new column, Alembic gives us a small, repeatable migration that records exactly how the schema should change.

```
class Document(Base):
    __tablename__ = "documents"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    title: Mapped[str] = mapped_column(
        String(120)
    )

    content: Mapped[str] = mapped_column(
        Text
    )

    priority: Mapped[int] = mapped_column(
        default=1
    )
```

documents

id	title	content	priority
----	-------	---------	----------

Now imagine we want to add another column:

```
id
title
content
priority
description ← NEW
```

Schema changes become versioned, repeatable database migrations



Figure 4.1 — A SQLAlchemy model change becomes a versioned Alembic migration before the SQLite schema changes.

Schema: is the structure of the database

Migration: is a recorded and organized step to change the database from a schema **version** to another.

VERSION 1

```
documents
├── id
├── title
├── content
└── priority
```

VERSION 2

```
documents
├── id
├── title
├── content
├── priority
└── description
```

```
# example — from Lecture 3 to Lecture 4
LECTURE 3 documents(id, title, content, priority)
MODEL CHANGE add optional description
MIGRATION    add description to the existing documents table
RESULT       the same documents.db moves to the new schema
```

PRO TIP `create_all()` is fine for the first beginner table. For later versioned schema changes, use a migration instead of hoping the existing database updates itself.

02

Connect Alembic to the Simple Project

Minimal Alembic setup

```
# install + initialize
pip install alembic
python -m alembic init alembic

# alembic.ini
sqlalchemy.url = sqlite:///documents.db

# alembic/env.py
from database import Base
import models

target_metadata = Base.metadata
```

Keep the same SQLite file from Lecture 3. To make Alembic easy to import, place engine + Base in database.py and the Document ORM class in models.py. Alembic reads Base.metadata to understand the mapped tables. We still keep database access inside routes simple with Session(engine).

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase

DATABASE_URL = "sqlite:///documents.db"

engine = create_engine(
    DATABASE_URL
)

class Base(DeclarativeBase):
    pass
```

```
#models.py

from database import Base
from sqlalchemy import String, Text
from sqlalchemy.orm import Mapped, mapped_column
from database import engine

class Document(Base):
    __tablename__ = "documents"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    title: Mapped[str] = mapped_column(
        String(120)
    )

    content: Mapped[str] = mapped_column(
        Text
    )

    priority: Mapped[int] = mapped_column(
        default=1
    )

Base.metadata.create_all(engine)
```

✓ QUICK CHECK

Q1. What does `target_metadata = Base.metadata` give Alembic?

Answer: The SQLAlchemy table metadata that Alembic compares with the database schema.

Q2. Why import models in `env.py`?

Answer: So the Document mapping is registered in `Base.metadata` before `autogenerate` runs.

03 Create & Apply One Migration

Now change the ORM model in one small way: add an optional description column. Autogenerate compares the updated model metadata with the existing `documents.db` schema and creates a migration script. Review that script, then apply it with `upgrade head`.

Model change + migration commands

```
# models.py – add one optional column
from sqlalchemy import String
```

```
description: Mapped[str | None] = mapped_column(
    String(250),
    nullable=True
```

```
# schemas.py
from pydantic import BaseModel, ConfigDict, Field
class DocumentCreate(BaseModel):
```

```
    title: str = Field(
        min_length=3,
        max_length=120
    )
```

```
    content: str = Field(
        min_length=1
    )
```

```
    priority: int = Field(
        default=1,
        ge=1,
        le=5
    )
```

```
    description: str | None = Field(
        default=None,
        max_length=250
    )
```

```
class DocumentResponse(BaseModel):
```

```
    id: int
    title: str
    content: str
    priority: int
    description: str | None = None
```

```
    model_config = ConfigDict(
        from_attributes=True
    )
```

And then we make

```
routes/__init__.py
```

to make routes explicit package for python

Then we put all **CRUD** operations in one separate file:

```
routes/documents.py

from fastapi import (
    APIRouter,
    HTTPException,
    status,
)
from sqlalchemy.orm import Session
from database import engine
from models import Document
from schemas import (
    DocumentCreate,
    DocumentResponse,
)

router = APIRouter(
    prefix="/documents",
    tags=["documents"],
)

# =====
# GET ALL
# =====

@router.get(
    "",
    response_model=list[DocumentResponse],
    summary="List all documents",
)
def list_documents():
    with Session(engine) as db:
        documents = db.query(
            Document
        ).all()

        return documents

# =====
# CREATE
# =====

@router.post(
    "",
    response_model=DocumentResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create a document",
)
def create_document(
```

```

    payload: DocumentCreate
):
    with Session(engine) as db:
        doc = Document(
            title=payload.title,
            content=payload.content,
            priority=payload.priority,
            description=payload.description,
        )
        db.add(doc)
        db.commit()
        db.refresh(doc)
        return doc

# =====
# GET ONE
# =====

@router.get(
   ("/{document_id}",
    response_model=DocumentResponse,
    summary="Get one document",
    responses={
        404: {
            "description": "Document not found"
        }
    },
)
def get_document(
    document_id: int
):
    with Session(engine) as db:
        doc = db.get(
            Document,
            document_id
        )
        if doc is None:
            raise HTTPException(
                status_code=404,
                detail="Document not found"
            )
        return doc

# =====
# UPDATE
# =====

@router.put(
   ("/{document_id}",
    response_model=DocumentResponse,
    summary="Update a document",

```

```

        responses={
            404: {
                "description": "Document not found"
            }
        },
    )
def update_document(
    document_id: int,
    payload: DocumentCreate,
):
    with Session(engine) as db:

        doc = db.get(
            Document,
            document_id
        )

        if doc is None:
            raise HTTPException(
                status_code=404,
                detail="Document not found"
            )

        doc.title = payload.title
        doc.content = payload.content
        doc.priority = payload.priority
        doc.description = payload.description

        db.commit()

        db.refresh(doc)

        return doc

# =====
# DELETE
# =====

@router.delete(
   ("/{document_id}",
    summary="Delete a document",
    responses={
        404: {
            "description": "Document not found"
        }
    },
)
def delete_document(
    document_id: int
):
    with Session(engine) as db:

        doc = db.get(
            Document,
            document_id
        )

```

```

    if doc is None:
        raise HTTPException(
            status_code=404,
            detail="Document not found"
        )

    db.delete(doc)

    db.commit()

    return {
        "message": "Document deleted"
    }

```

```

# example - simple migration flow
MODEL      add description to Document
REVISION   Alembic generates a migration file
REVIEW     read upgrade() before changing the database
UPGRADE    apply the migration to documents.db
CHECK      old rows still exist; the new column is available

```

```

# create, review, then apply the migration
python -m alembic revision --autogenerate -m "add description"
python -m alembic upgrade head
python -m alembic current

```

04 Organize Routes with APIRouter

After the database step works, move the Documents endpoints into an APIRouter. This changes code organization, not the CRUD idea from Lecture 3: each database endpoint can still open a direct with `Session(engine)` as db block.

- **main.py** — creates the FastAPI app and includes routers.
- **routes/documents.py** — contains the Documents HTTP endpoints.
- **database.py / models.py** — hold the engine, Base, and ORM model.

Same URLs, cleaner organization

```

# routes/documents.py
from fastapi import APIRouter
from sqlalchemy.orm import Session
from database import engine
from models import Document

router = APIRouter(prefix="/documents", tags=["documents"])

@router.get("")
def list_documents():
    with Session(engine) as db:
        return db.query(Document).all()

# main.py
from fastapi import FastAPI
from routes.documents import router as documents_router
app = FastAPI(title="Documents API")
app.include_router(documents_router)

```

PRO TIP *APIRouter changes where route code lives, not how the database works. Keep the direct `Session(engine)` pattern from Lecture 3 for now.*

05

Swagger UI, ReDoc & OpenAPI

Generated View	Default Location
Interactive Swagger UI	/docs
Reference-style ReDoc	/redoc
Machine-readable OpenAPI schema	/openapi.json
Request / response shapes	Pydantic models
Summaries / status / errors	Route decorator metadata
Application assembly	main.py

Document expected responses

```
@router.get(
   ("/{document_id}",
    response_model=DocumentResponse,
    summary="Get one document",
    responses={404: {"description": "Document not found"}},
)
def get_document(document_id: int):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        return doc
```

✓ QUICK CHECK

Q1. What is /docs?

Answer: FastAPI's generated interactive Swagger UI.

Q2. Does APIRouter require us to replace Session(engine)?

Answer: No. In this simplified course version, the same direct Session block can stay inside the router endpoint.

Work through these on your own before checking the marked answers.

Practice Question 1

Why do we introduce Alembic after Lecture 3?

- A. To version changes to an existing database schema
- B. To replace FastAPI
- C. To store HTML pages
- D. To remove Pydantic

Practice Question 2

What does `target_metadata = Base.metadata` help Alembic understand?

- A. The mapped SQLAlchemy tables and columns
- B. The browser history
- C. Uploaded files only
- D. Uvicorn reload settings

Practice Question 3

When routes move to `APIRouter`, what happens to the Lecture 3 `Session(engine)` idea?

- A. It can stay the same inside each database endpoint
- B. It must become PostgreSQL
- C. It must be deleted
- D. It becomes a Pydantic model

Practice Question 4

Where is Swagger UI available by default?

- A. `/docs`
- B. `/database`
- C. `/router`
- D. `/sqlite`

Practice Question 5

Why should an autogenerated Alembic migration be reviewed before upgrade?

- A. It contains schema-changing operations that should be verified
- B. It always deletes Pydantic models
- C. It is only documentation text
- D. It cannot affect the database

Answers

Practice Question 1: A ✓ — Alembic gives existing database schema changes a repeatable versioned history.

Practice Question 2: A ✓ — Alembic uses SQLAlchemy metadata to understand the mapped schema.

Practice Question 3: A ✓ — `APIRouter` changes route organization; the simple `Session(engine)` database pattern can remain.

Practice Question 4: A ✓ — FastAPI exposes Swagger UI at `/docs` by default.

Practice Question 5: A ✓ — Autogenerate is helpful, but schema-changing code should still be reviewed.